

Eugene Moshchyn

Dr. Shangqing Zhao

CS 5173, Section 1

March 13, 2026

Homework 3

Part 1 — Questions

Problem 1 What are the properties of hash functions?

Solution

Hash function has the following properties:

- (a) Performance: Easy to compute $H(m)$
Own Explanation: It should be efficient and not computationally intensive to compute.
- (b) One-Way: Given $H(m)$ but not m , it's **computationally infeasible** to find m
Own Explanation: The practical approach is essentially brute force (guess messages and hash them) until a match is found.
- (c) Weak Collision Resistance: Given $H(m)$, it's **computationally infeasible** to find m' such that $H(m') = H(m)$ *Own Explanation*: If we already have a valid message and its hash, we should not be able to find a second, altered, message that produces that exact same hash.
- (d) Strong Collision Resistance: **computationally infeasible** to find m_1, m_2 such that $H(m_1) = H(m_2)$ *Own Explanation*: Due to fixed output lengths, collisions can mathematically exist, but it should be virtually impossible to find two different messages that result in the same hash.

Note

The following question is identical to the one in Homework 2.

Problem 2 Bob designed a new scheme to detect whether a message is modified by an attacker (protecting data integrity). The scheme is to append to each message a SHA-256 hash of that message. Explain the vulnerability of Bob's design.

Solution

There are significant vulnerabilities in this approach, primarily because an unkeyed hash function cannot guarantee data integrity against an active attacker:

- (a) **Origin Authentication:** Bob's scheme provides no way to verify the identity of the sender. Because no secret key is required to generate the hash, any attacker can create a message-and-hash pair from scratch and send it to the receiver while masquerading as a legitimate user.
- (b) **Integrity:** Since SHA-256 is a public unkeyed algorithm, an active attacker can intercept the message in transit, alter its contents, recalculate the SHA-256 hash for the modified message, and forward the new payload to the receiver. The receiver will compute the hash, see that it matches the appended hash, and incorrectly assume the message was unmodified.

Problem 3

Bob designed another new scheme to detect whether a message is modified by an attacker (protecting data integrity). The scheme is to append to each message a SHA-256 hash of that message, then encrypt the whole appended message using AES with a known key to the public. Explain the vulnerability of Bob's design.

Solution

The primary vulnerability is that the scheme lacks a secret element (such as a private key or a shared secret) to prove the message's origin.

Because the AES key is public, an attacker can perform a man-in-the-middle attack by decrypting the message, altering the content, and computing a new SHA-256 hash for the modified data. They can then re-encrypt the new **message + hash** bundle using the public key.

Since SHA-256 is a public algorithm and the AES key is known, the receiver's integrity check will succeed even if the message was changed. This scheme fails to provide authenticity, which is a prerequisite for true integrity in an adversarial environment.

Problem 4

A company is selling an artifact worth millions of dollars. Many interested buyers are in the bidding process, which works as follows. Each buyer sends the information of his/her final number (e.g., \$1,000,000.00) to the company to show his/her commitment. And the number must be kept confidential from other buyers and even the company before the deadline. After the deadline, all numbers are released to all buyers and the company, then the buyer with the highest number wins. Design a HASH FUNCTION based scheme to secure the bidding process. Describe steps a buyer and/or the company should do.

Solution

To ensure the bidding process is secure, confidential, and verifiable, the system should implement a Cryptographic Commitment Scheme consisting of a “Commit” phase and a “Reveal” phase.

Phase 1: Commit

- (a) An interested buyer decides on their bid amount (B). They then generate a large, cryptographically secure random number, salt or nonce (K).
- (b) The buyer hashes their bid amount combined with the nonce: $C = H(B, K)$. This resulting hash (C) is their “commitment”.
- (c) The buyer sends only their commitment (C) alongside their unique identification (such as, system ID, full name, driver’s license) to the company.

Phase 2: Reveal

- (a) Once the deadline passes, all buyers publicly release their original plaintext bid amount (B) and their secret nonce (K) to the company and all other buyers.
- (b) The company (and any participating buyer) verifies the bids by recomputing the hash using the revealed data: $H(B, K)$

Note

To ensure no bias during the selection process based on buyer identification, the company could implement a blinded approach where the buyer’s ID is also hashed or anonymized during the Commit phase. Furthermore, to protect network-level privacy, buyers could submit their commitments via a reverse proxy or an anonymity network (like Tor/onion routing) to hide their IP addresses, ensuring that the company cannot identify or profile bidders prior to the Reveal phase.

Problem 5

Write your **OWN** code of computing SHA1 and compute the following hashes.

- (a) OU
- (b) Computer Science
- (c) Crypto

You can verify the results of your code using: <http://www.sha1-online.com/>

Submit your code and screenshots of your results. Notes:

- (a) You cannot just directly call a SHA1 function/algorithm in a program language. You need to code the internals of SHA1.
- (b) Writing the correct SHA1 code might be challenging. You need to be quite familiar with the bit operations in programming languages like C. You can use SHA1 code

available online as reference. But you cannot directly copy the code from Internet since OU plagiarism system can detect it.

Solution

```

1  /**
2   * @file      sha1.h
3   * @author    Eugene Moshchyn
4   * @studentID 113-683-484
5   * @course    CS 5173 - Computer Security
6   * @date      Mar 13th, 2026
7   * @brief     SHA1 Algorithm Implementation
8   */
9
10 #pragma once
11
12
13 #include <iostream>
14 #include <vector>
15 #include <string>
16 #include <cstdint>
17 #include <iomanip>
18 #include <sstream>
19
20 /*
21  * Helper function
22  * Source: https://en.wikipedia.org/wiki/Circular\_shift#Implementing\_circular\_shifts
23  */
24 /// Source:
25 inline uint32_t leftrotate(uint32_t val, size_t count)
26 {
27     return (val << count) | (val >> (32 - count));
28 }
29
30
31 /*
32  * SHA1 Implementation
33  * Source: https://en.wikipedia.org/wiki/SHA-1#SHA-1\_pseudocode
34  */
35 std::string sha1(const std::string& message)
36 {
37     // Initialize variables
38     uint32_t h0 = 0x67452301,
39             h1 = 0xEFCDAB89,
40             h2 = 0x98BADCFE,
41             h3 = 0x10325476,
42             h4 = 0xC3D2E1F0;
43
44     uint64_t message_bitlength = message.length() * 8;
45
46
47     // Pre-processing

```

```

48     std::vector<uint8_t> padded_message(message.begin(), message.end())
49     ;
50     padded_message.push_back(0x80);           // append the bit '1' to
51     the message
52     while ((padded_message.size() % 64) != 56) // append '0', until
53     message length is 448 bits
54     {
55         padded_message.push_back(0x00);
56     }
57     for (int i = 7; i >= 0; --i) // append message_bitlength
58     {
59         padded_message.push_back(static_cast<uint8_t>((
60         message_bitlength >> (i * 8)) & 0xFF));
61     }
62     // Process the message in successive 512-bit chunks
63     for (size_t offset = 0; offset < padded_message.size(); offset +=
64     64)
65     {
66         std::vector<uint32_t> w(80, 0);
67
68         // Break chunk into sixteen 32-bit big-endian words
69         for (int i = 0; i < 16; ++i)
70         {
71             w[i] = (padded_message[offset + i * 4] << 24) |
72             (padded_message[offset + i * 4 + 1] << 16) |
73             (padded_message[offset + i * 4 + 2] << 8) |
74             (padded_message[offset + i * 4 + 3]);
75         }
76         // Message schedule: extend the sixteen 32-bit words into
77         eighty 32-bit words
78         for (int i = 16; i < 80; ++i)
79         {
80             w[i] = leftrotate(w[i - 3] ^ w[i - 8] ^ w[i - 14] ^ w[i -
81             16], 1);
82         }
83         // Initialize hash value for this chunk
84         uint32_t a = h0;
85         uint32_t b = h1;
86         uint32_t c = h2;
87         uint32_t d = h3;
88         uint32_t e = h4;
89
90         // Main loop
91         for (int i = 0; i < 80; ++i)
92         {
93             uint32_t f = 0,
94             k = 0;

```

```

95     switch (i / 20)
96     {
97         case 0: // 0 ... 19
98             f = (b & c) | ((~b) & d);
99             k = 0x5A827999;
100            break;
101         case 1: // 20 ... 39
102             f = b ^ c ^ d;
103             k = 0x6ED9EBA1;
104            break;
105         case 2: /// 40 ... 59
106             f = (b & c) | (b & d) | (c & d);
107             k = 0x8F1BBCDC;
108            break;
109         case 3: /// 60 ... 79
110             f = b ^ c ^ d;
111             k = 0xCA62C1D6;
112            break;
113         default:
114            break;
115     }
116
117     uint32_t temp = leftrotate(a, 5) + f + e + k + w[i];
118     e = d;
119     d = c;
120     c = leftrotate(b, 30);
121     b = a;
122     a = temp;
123 }
124
125 // Add this chunk's hash to result so far
126 h0 += a;
127 h1 += b;
128 h2 += c;
129 h3 += d;
130 h4 += e;
131 }
132
133 // Produce the final hash value (big-endian) as a 160-bit number
134 std::stringstream result;
135 result << std::hex << std::setfill('0')
136        << std::setw(8) << h0
137        << std::setw(8) << h1
138        << std::setw(8) << h2
139        << std::setw(8) << h3
140        << std::setw(8) << h4;
141
142 return result.str();
143 }

```

Listing 1: SHA1 Function, located in `sha1.h`

```

1  /**
2   * @file      main.cpp
3   * @author    Eugene Moshchyn

```

```

4  * @studentID 113-683-484
5  * @course    CS 5173 - Computer Security
6  * @date      Mar 13th, 2026
7  * @brief     Main Function that utilizes SHA1
8  **/
9
10
11 #include "sha1.h"
12
13
14 int main() {
15     std::vector<std::string> messages = {
16         "OU",
17         "Computer Science",
18         "Crypto"
19     };
20
21     for (auto message : messages)
22     {
23         std::cout << "Message:    " << message << std::endl;
24         std::cout << "SHA-1 Hash: " << sha1(message) << std::endl;
25         std::cout << std::endl;
26     }
27
28     return 0;
29 }

```

Listing 2: Main function, located in `main.cpp`

Here are the screenshots proving the successful compilation of the program on a Linux machine using the `g++` compiler, alongside the correct output of the hashes.

The screenshot shows a Visual Studio Code editor with a dark theme. The Explorer pane on the left shows a project named 'main' containing 'main.cpp' and 'sha1.h'. The main editor window displays a message: 'The file is not displayed in the text editor because it is either binary or uses an unsupported text encoding.' Below this, the Terminal pane shows the following output:

```

$ g++ main.cpp sha1.h -o main
$ ./main
Message:    OU
SHA-1 Hash: d29a663de815adfb49ac20b6f2994284c409fd2c

Message:    Computer Science
SHA-1 Hash: 60b8102bd020e99e3357c3d6079d6e1421c5ce57

Message:    Crypto
SHA-1 Hash: e849494484ed2e3c1a93babc3e347d2e98ac8604

```

Figure 1: Compilation of SHA1 program with specified inputs of `OU`, `Computer Science`, and `Crypto`

Here are the screenshots proving that the implemented algorithm correctly computes the hashes of the specified inputs and matches the ones calculated at <http://www.sha1-online.com/>.

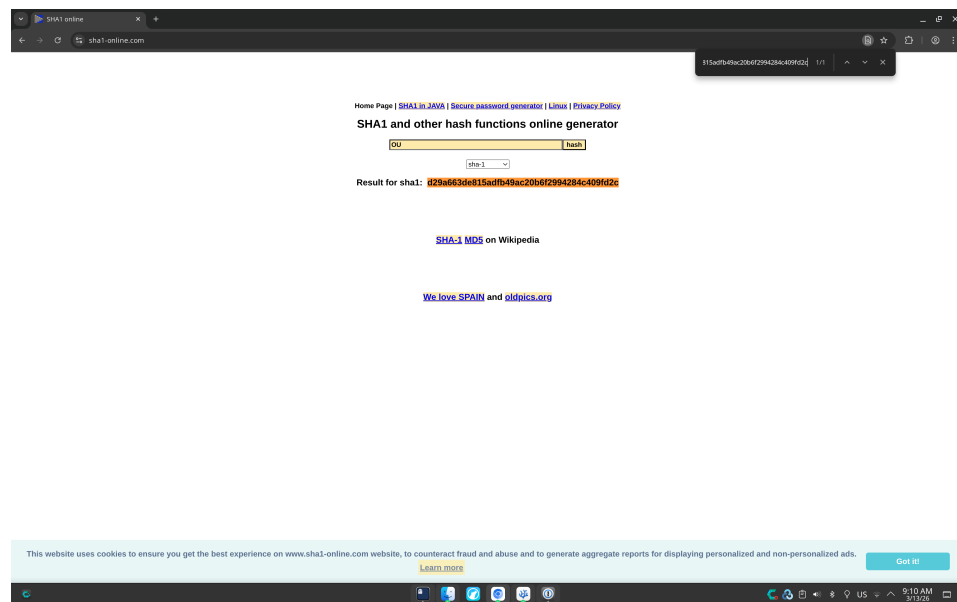


Figure 2: Correct calculation of hash for **OU**

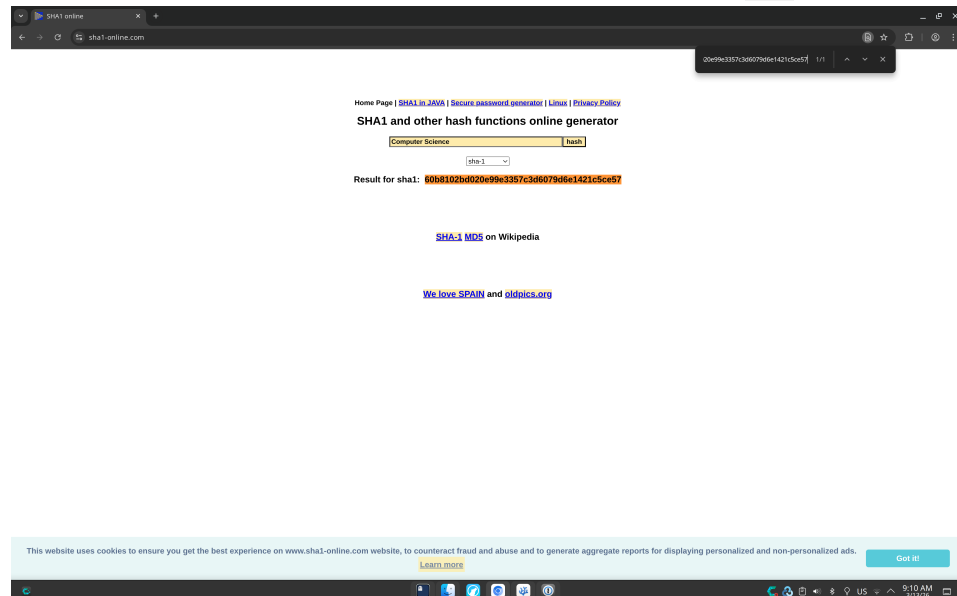
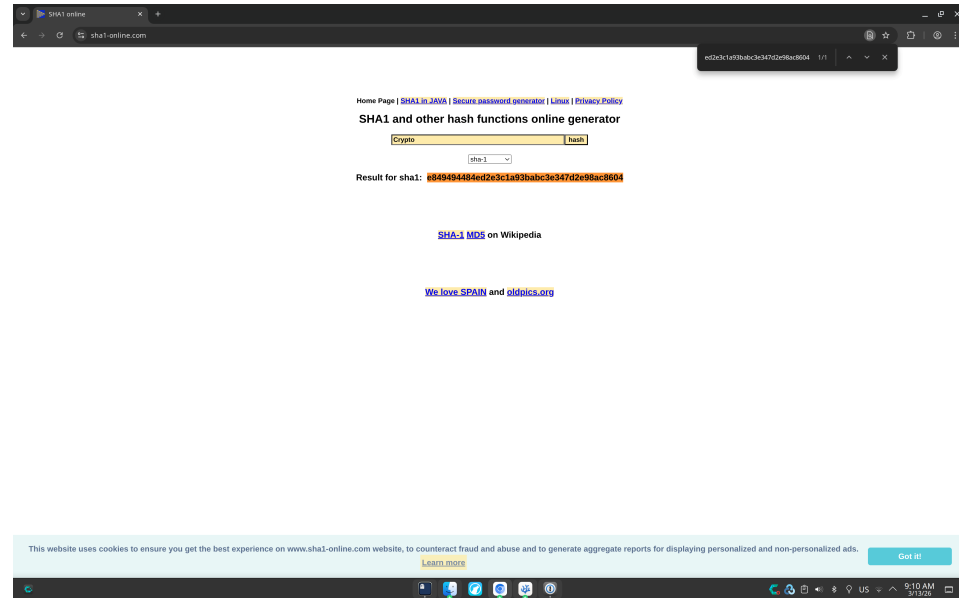


Figure 3: Correct calculation of hash for **Computer Science**

Figure 4: Correct calculation of hash for **Crypto****Problem 6**

Write a computer program to implement the extended Euclid's algorithm, use your code to compute the following. Submit your code and results.

- GCD(10012012,2314213)
- GCD(28176412,29108188)
- GCD(38172,23812188)
- The multiplicative inverse of 12091 mod 24123123.
- The multiplicative inverse of 28173928 mod 129182771.
- GCD(381723029127918237717233210002,23812188332813212739187261)
for the graduate session only.

Solution

```

1  """
2  @file      gcd.py
3  @author    Eugene Moshchyn
4  @studentID 113-683-484
5  @course    CS 5173 - Computer Security
6  @date      Mar 13th, 2026
7  @brief     GCD and Mod Inverse Functions
8  """
9
10 def extended_gcd(a, b):
11     """
12     Implements the Extended Euclidean Algorithm.
13     Returns a tuple (gcd, x, y) such that a*x + b*y = gcd.
14     """

```

```

15     old_r, r = a, b
16     old_s, s = 1, 0
17     old_t, t = 0, 1
18
19     while r != 0:
20         quotient = old_r // r
21         old_r, r = r, old_r - quotient * r
22         old_s, s = s, old_s - quotient * s
23         old_t, t = t, old_t - quotient * t
24
25     return old_r, old_s, old_t
26
27 def mod_inverse(a, m):
28     """
29     Computes the multiplicative inverse of a modulo m using the
30     Extended
31     Euclidean Algorithm. Returns the inverse, or None if no inverse
32     exists
33     (i.e., gcd(a, m) != 1).
34     """
35     gcd, x, _ = extended_gcd(a, m)
36     if gcd != 1:
37         return None # Modular inverse does not exist
38     return x % m

```

Listing 3: GCD Functions, located in `gcd.py`

```

1  """
2  @file      main.py
3  @author    Eugene Moshchyn
4  @studentID 113-683-484
5  @course    CS 5173 - Computer Security
6  @date      Mar 13th, 2026
7  @brief     Main Function
8  """
9
10
11 from tabulate import tabulate
12 from gcd import ( extended_gcd, mod_inverse )
13
14
15 def main():
16     print("--- Extended Euclidean Algorithm Results ---")
17
18     data = [
19         [
20             "a",
21             "GCD",
22             10012012,
23             2314213,
24             extended_gcd(10012012, 2314213)[0]
25         ],
26         [
27             "b",
28             "GCD",

```

```

29         28176412,
30         29108188,
31         extended_gcd(28176412, 29108188)[0]
32     ],
33     [
34         "c",
35         "GCD",
36         38172,
37         23812188,
38         extended_gcd(38172, 23812188)[0]
39     ],
40     ["d",
41         "Inverse",
42         12091,
43         24123123,
44         mod_inverse(12091, 24123123)
45     ],
46     ["e",
47         "Inverse",
48         28173928,
49         129182771,
50         mod_inverse(28173928, 129182771)
51     ],
52     ["f",
53         "GCD",
54         381723029127918237717233210002,
55         23812188332813212739187261,
56         extended_gcd(381723029127918237717233210002,
23812188332813212739187261)[0]
57     ]
58 ]
59
60 headers = ["Item", "Type", "Input A", "Input B / Mod", "Result"]
61
62 print(tabulate(data, headers=headers, tablefmt="grid"))
63
64
65 if __name__ == '__main__':
66     main()

```

Listing 4: Main Function, located in `main.py`

Here is a screenshot demonstrating the successful execution and correct output of the program:

```

python -m venv venv
source venv/bin/activate
pip install tabulate
Collecting tabulate
  Downloading tabulate-0.10.0-py3-none-any.whl.metadata (40 kB)
  Downloading tabulate-0.10.0-py3-none-any.whl (39 kB)
Installing collected packages: tabulate
Successfully installed tabulate-0.10.0

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
python rs3in.py
--- Extended Euclidean Algorithm Results ---


| Item | Type    | Input A                        | Input B / Mod              | Result    |
|------|---------|--------------------------------|----------------------------|-----------|
| a    | GCD     | 18012012                       | 2314213                    | 1         |
| b    | GCD     | 28176412                       | 29188188                   | 4         |
| c    | GCD     | 38172                          | 23812188                   | 12        |
| d    | Inverse | 12091                          | 24123123                   | 9187576   |
| e    | Inverse | 28179928                       | 129182771                  | 181412588 |
| f    | GCD     | 381723829127918237717233218892 | 23812188332813212739187261 | 1         |


```

Figure 5: Execution of Python Script

Note

Python supports arbitrary-precision integers (bigints) natively, allowing for seamless large-number manipulation. Similarly, Java provides a built-in **BigInteger** class which facilitates a similarly straightforward implementation. On the other hand, languages without native bigint support, such as C or C++, would require a custom library or class and additional logic to handle such values correctly.

Note

The `tabulate` library was used to generate the formatted output tables. If you intend to run this script locally, please ensure the package is installed in your Python environment (e.g., via `pip install tabulate`).

Problem 7 Prove that $a = n - 1$ is always a solution to $a^2 \equiv 1 \pmod{n}$.

Solution

We want to show that if $a = n - 1$, then

$$a^2 \equiv 1 \pmod{n}.$$

Substituting $a = n - 1$, it suffices to prove that

$$(n - 1)^2 \equiv 1 \pmod{n}.$$

Expanding the square gives

$$(n - 1)^2 = n^2 - 2n + 1.$$

Now reduce modulo n :

$$n^2 - 2n + 1 \equiv 0 - 0 + 1 \equiv 1 \pmod{n},$$

since both n^2 and $2n$ are multiples of n .

Therefore,

$$(n - 1)^2 \equiv 1 \pmod{n},$$

so $a = n - 1$ is always a solution to $a^2 \equiv 1 \pmod{n}$. ■

Problem 8 What are the differences between symmetric and asymmetric key crypto?

Solution

Symmetric-key cryptography uses the same shared secret key for both encryption and decryption.

Asymmetric-key cryptography uses two related keys: a public key and a private key.

Symmetric cryptography is usually faster, while asymmetric cryptography is slower but useful for secure key exchange, authentication, and digital signatures.

Problem 9 Explain the feasibility and security of RSA.

Solution

The **feasibility** of RSA relies on the fact that computers can easily and quickly multiply two very large prime numbers together to generate cryptographic keys, making encryption and decryption fast enough for practical use. The **security** of RSA relies on the fact that reversing this process, factoring that massive resulting number back into its original primes, is computationally infeasible. Without the private key, it would take modern computers an impossibly long time to crack the encryption.

Problem 10 Alice and Bob use the Diffie-Hellman to create two keys k_1 and k_2 . In generating k_1 , Alice selects a random number x and computes $S_A = g^x \pmod{p}$. Bob selects a random number y and computes $S_B = g^y \pmod{p}$. Alice and Bob exchange S_A and S_B , and $k_1 = g^{xy} \pmod{p}$. In generating k_2 , Alice chooses another random number x' but Bob still uses the same random number y . Suppose an eavesdropper knows that x and x' differ each other by t (i.e., either $x - x' = t$ or $x' - x = t$). Given the knowledge of k_1 and a pair of plaintext M and ciphertext C encrypted by k_2 , can the eavesdropper find out k_2 ?

Solution

Since x and x' differ by t , there are only two possibilities: $x' = x + t$ or $x' = x - t$.

The eavesdropper already knows $k_1 = g^{xy} \bmod p$ and Bob's public share $S_B = g^y \bmod p$. They can plug these known values in to compute the two potential candidates for the second key ($k_2 = g^{x'y} \bmod p$):

Candidate 1 (if $x' = x + t$):

$$k_2 = g^{(x+t)y} = g^{xy} \cdot (g^y)^t = k_1 \cdot (S_B)^t \bmod p$$

Candidate 2 (if $x' = x - t$):

$$k_2 = g^{(x-t)y} = g^{xy} \cdot (g^y)^{-t} = k_1 \cdot (S_B)^{-t} \bmod p$$

The eavesdropper calculates both candidates. To identify the real k_2 , they just test which candidate successfully encrypts the known plaintext M to produce the matching ciphertext C .

Problem 11

Alice designs a double-RSA cipher. She first generates two secret primes p and q , and compute $n = p * q$, then choose two public encryption exponents e_1 and e_2 that are relatively prime to $\phi(n)$. So $\langle e_1, e_2, n \rangle$ becomes the public key. She tells people to encrypt message M by computing $C_1 = M^{e_1} \bmod n$ and then $C = C_1^{e_2} \bmod n$, finally sending just C to her.

- Show the decryption process (i.e., how Alice can obtain the plaintext M from the final ciphertext C).
- Is the double-RSA cipher more secure, less secure, or just as secure as the regular RSA cipher with the same modulus n but only one encryption exponent? Why?
- Charlie got Alice's instructions confused, and encrypt message M for Alice using e_1 and e_2 in the reverse order (i.e., Charlie uses $C_1 = M^{e_2} \bmod n$ then $C = C_1^{e_1} \bmod n$). What would happen when Alice, unaware of Charlie's error, tries to decipher the ciphertext using her usual procedure?

Solution

Subproblem A

In order to decrypt the message from the ciphertext, Alice first needs to calculate n and $\phi(n)$ using her secret primes p and q :

$$\begin{aligned} n &= p \cdot q \\ \phi(n) &= (p - 1)(q - 1) \end{aligned}$$

The final ciphertext C encrypted using the public exponents e_1 and e_2 expands as follows:

$$\begin{aligned} C &= C_1^{e_2} \bmod n \\ &= (M^{e_1})^{e_2} \bmod n \\ &= M^{e_1 e_2} \bmod n \end{aligned}$$

Since both e_1 and e_2 are public, we can represent their product as a single public exponent $e = e_1 e_2$. Because Alice knows $\phi(n)$, she can calculate the corresponding private decryption exponent d :

$$\begin{aligned} d &= e^{-1} \bmod \phi(n) \\ &= (e_1 e_2)^{-1} \bmod \phi(n) \end{aligned}$$

Based on the properties of RSA, we know that $e \cdot d \equiv 1 \pmod{\phi(n)}$. Therefore, Alice can obtain the plaintext M from the incoming ciphertext C using the standard RSA decryption procedure:

$$\begin{aligned} M &= C^d \bmod n \\ &= (M^e)^d \bmod n \\ &= M^{ed} \bmod n \\ &= M^1 \bmod n \\ &= M \end{aligned}$$

Subproblem B

The double-RSA cipher is just as secure as a regular RSA cipher with the same modulus n . It is neither more nor less secure. Because mathematically, the “double” encryption collapses into a single standard RSA encryption. By computing $(M^{e_1})^{e_2} \bmod n$, you are simply performing a standard RSA encryption using a new public exponent where

$$e = e_1 \cdot e_2$$

The security of RSA relies on the difficulty of factoring the modulus n . Since n remains exactly the same, the fundamental mathematical hardness of the problem has not changed. An attacker doesn’t need to break two layers of RSA; they just need to break one layer of RSA that happens to use a slightly larger public exponent.

Subproblem C

If Charlie encrypts in the reverse order, he first uses e_2 and then e_1 . His math looks like this:

$$C_{\text{Charlie}} = (M^{e_2})^{e_1} \bmod n = M^{e_2 \cdot e_1} \bmod n$$

Because basic multiplication is commutative, Charlie’s resulting ciphertext is exactly identical to the ciphertext he would have generated if he had followed the instructions correctly.

Therefore, when Alice tries to decipher it using her usual procedure, she will successfully recover the plaintext M without ever realizing Charlie made an error.

Part 2 — Lab

Problem 12 In this lab exercise, you will work with a BMP image file named **pic_original.bmp** and explore how encryption affects visual data. Your task is to encrypt the image using OpenSSL in both ECB (Electronic Code Book) and CBC (Cipher Block Chaining) modes. For example, you might use commands like **openssl enc -aes-256-ecb -in pic_original.bmp -out pic_ecb.enc** for ECB mode and a similar command with **-aes-256-cbc** for CBC mode.

Because BMP files rely on the first 54 bytes as header information to display correctly, you must extract the header from the original image and combine it with the encrypted data: extract the header with **head -c 54 pic_original.bmp > header**, extract the encrypted body from byte 55 onward using **tail -c +55 pic_ecb.enc > body**, and then concatenate them via **cat header body > pic_ecb.bmp** (repeat for the CBC-encrypted file). Finally, open the resulting BMP files using an image viewer and observe any visible patterns or clues that might reveal aspects of the original picture. Please submit screenshots of your terminal commands, the resulting images, and a brief report discussing your observations and any differences between the ECB and CBC encryption outcomes.

For Linux and macOS, you can directly use the head and tail commands. For Windows users, you have two options: either run these commands via **WSL** (Windows Subsystem for Linux) or use a hex editor (such as HxD) to manually copy the first 54 bytes from **pic_original.bmp** and overwrite the first 54 bytes in each encrypted file. Then, save the modified files as **pic_ecb.bmp** and **pic_cbc.bmp**.

Solution

The following commands were executed to perform the encryption. The 256-bit encryption key used for this process was:

```
Kr6X05YCMX1LEZnmPpfK5KE0pyA2M00Uw2CA1J3YqYA7LLMs90 :
```

```

$ openssl --version
OpenSSL 3.6.1 27 Jan 2026 (Library: OpenSSL 3.6.1 27 Jan 2026)
$ head -c 54 pic_original.bmp > header
$ openssl enc -aes-256-ecb -in pic_original.bmp -out pic_ecb.enc
enter AES-256-ECB encryption password:
Verifying - enter AES-256-ECB encryption password:
** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
$ openssl enc -aes-256-cbc -in pic_original.bmp -out pic_cbc.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
$ tail -c +55 pic_ecb.enc > pic_ecb.enc.body
$ tail -c +55 pic_cbc.enc > pic_cbc.enc.body
$ cat header pic_ecb.enc.body > pic_ecb.bmp
$ cat header pic_cbc.enc.body > pic_cbc.bmp

```

Figure 6: Execution of OpenSSL encryption commands

Here is the original, unencrypted image for reference:



Figure 7: Original unencrypted image

Here is the resulting image after encryption using Electronic Codebook (ECB) mode:

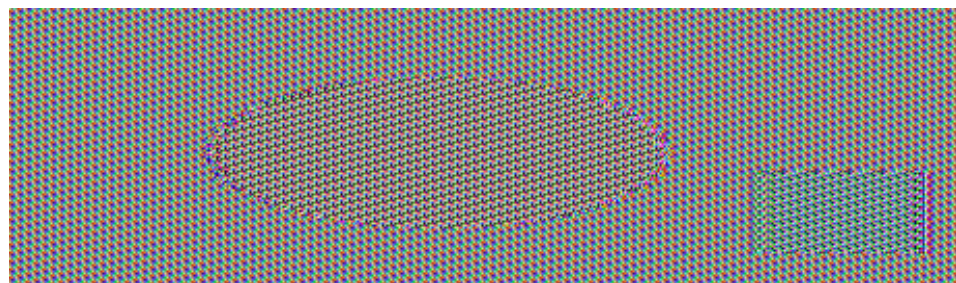


Figure 8: Image encrypted using ECB mode

Here is the resulting image after encryption using Cipher Block Chaining (CBC) mode:



Figure 9: Image encrypted using CBC mode

Analysis:

As observed in Figure 8, encrypting the image using ECB mode fails to mask the visual patterns of the original file. This occurs because ECB mode is deterministic; it encrypts identical plaintext blocks into identical ciphertext blocks. Consequently, regions of uniform color in the original image translate directly into repeating ciphertext blocks, leaking the underlying structural data.

On the other hand, the image encrypted using CBC mode (Figure 9) successfully hides all distinct visual characteristics. By utilizing an Initialization Vector (IV) and XORing each plaintext block with the previous ciphertext block before encryption, CBC diffuses the data. This eliminates block repetition, resulting in a ciphertext output that appears as high-entropy, pseudorandom noise.